

ParcelPilot AI Support Agent

Interview Study Guide — complete project reference

An AI customer-support + internal-ops agent over a deliberately imperfect corpus (6 PDFs + a workbook) for a B2B logistics company. Built as a CalQuity AI-Engineer take-home. The graded core is handling conflicting, deprecated, and wrong sources correctly — not retrieval or UI polish.

ELEVATOR PITCH One agent, two personas (customer / internal-ops). A deterministic source-precedence resolver and deterministic calculators decide every fee, credit and SLA — the LLM orchestrates and explains but never does the math or decides eligibility. Every answer cites its governing source; historical/deprecated sources are never used as authority. Stack: FastAPI + SQLite + hybrid BM25/local-embeddings retrieval + a hand-written tool loop on Groq (Gemini fallback), React front end, single-container hosting.

The two planted traps (the heart of the assessment)

Trap	Wrong historical answer	Correct answer	Why
Cancellation (TKT-450, Northstar)	"INR 250 fee applies after 30 min"	No fee, ever, before pickup	Northstar's contract waives the fee; contract > SOP. Wrong even on its own date.
Bulk upload (TKT-451, LumenWorks)	"Growth plan caps at 3,000 rows"	5,000 supported; 3,000 is a bug workaround	Product limit is 5,000; 3,000 is the KI-208 workaround threshold, not a plan cap.

Snapshot / frozen clock: 2026-08-16 11:00 Asia/Kolkata. Currency: INR. Live: <https://parcelpilot-keed.onrender.com> | Repo: [gurkirat309/ParcelPilot](#)

1. Problem & what is graded

ParcelPilot is a B2B logistics platform. Customers ask about entitlements, contract terms, cancellations, service credits, and SLAs, and report product issues. The support corpus is intentionally imperfect: one policy is deprecated, two customer agreements override general policy, and some historical ticket resolutions are wrong.

- Graded core: deliberately handling source authority / conflicts / uncertainty — not retrieval quality or UI.
- Must load and reason at runtime — no hard-coded IDs or answers (they test with other records).
- Two example questions: cancel ORD-1001 without a fee (Northstar); 3h-late carrier-fault credit.
- Two extra client problems — we addressed both: Problem 1 (proactive detection) and Problem 2 (trust & reliability).

2. The 10 non-negotiable rules (architecture invariants)

#	Rule
1	Embeddings are ALWAYS local (fastembed). Design for few, fat tool calls; agent loop capped at 8 iterations.
2	All LLM access goes through src/llm/client.py. No provider SDK calls anywhere else.
3	Time is frozen: the workbook snapshot is the only 'now'. datetime.now()/date.today() banned; a test greps for them.
4	Access control lives in the data layer. account_id comes from the session, never a model-set parameter.
5	State-changing tools never write on first call. They return a pending Proposal; execution is a separate confirm step.
6	All money/fee/credit/SLA math is deterministic Python, unit-tested. The model never does arithmetic or decides eligibility.
7	Deprecated docs excluded from default retrieval; retrievable only for 'what changed' queries.
8	No agent frameworks. Hand-written tool-calling loop. FastAPI + plain SDK calls only.
9	Never hard-code order IDs, account names, or example answers.
10	Never invent policy. If it is not in the sources, say so and escalate.

3. Architecture & request flow

```
React SPA --HTTP / SSE--> FastAPI
                        |- auth: bearer token -> Session (role, account_id server-side)
                        |- agent loop (hand-written, <=8 iters, streams tool steps)
                        |   |- LLM client (Groq primary | Gemini fallback)
                        |   '- tools:
                        |       - search_documents -> retrieval (BM25 + fastembed)
                        |       - get_order/account/ticket -> repository (scope-enforced)
                        |       - calc_* -> deterministic domain calculators
                        |       - create_escalation/update_ticket/... -> proposals
                        '- /ops/signals -> Problem-1 analytics
```

Layering (src/): config (frozen clock) - domain (policy+citations, precedence, calculators) - data (sqlite ingest, repository) - retrieval (chunks, hybrid index) - llm (client) - agent (tools, loop, proposals) - ops (signals) - api (app, auth).

- Backend: Python 3.13, FastAPI, SQLite (stdlib).
- Retrieval: rank_bm25 + fastembed BAAI/bge-small-en-v1.5 (local CPU, ~24 chunks).
- LLM: Groq openai/gpt-oss-120b primary; Gemini gemini-3.7-flash fallback; one client module.

- Frontend: React + Vite; streamed tool traces, confirmation cards, ops board.
- Hosting: single container (FastAPI serves built React); Dockerfile + render.yaml.

4. Source precedence & the conflict register (THE CORE)

A deterministic resolver ranks sources; the same ranking gates retrieval and resolves the terms the calculators use. Contracts are scope-gated — an account only ever sees its own contract.

Tier	Source	Role in an answer
1 (highest)	Customer's OWN agreement	Overrides policy, but only for its own account.
2	Current policy / SOP (v3, SOP v4)	The default floor when no contract applies.
3	Product Operations Guide	Capabilities, limits, known issues (KI-208, KI-211).
4	Historical ticket resolutions	CONTEXT ONLY. May be wrong. NEVER cited as a basis.
5 (lowest)	Deprecated docs (Policy v2)	Excluded by default; only for 'what changed' queries.

Sources at a glance

File	Type	Status	Effective	Binds
01 Support Policy v3	policy	CURRENT	2026-05-01	global
02 Support Policy v2	policy	DEPRECATED	2025-01-01	global (excluded)
03 Cancellation & Credit SOP v4	sop	CURRENT	2026-06-15	global
04 Product Ops Guide + Known Issues	guide	CURRENT	2026-08-14	global
05 Northstar Enterprise Agreement	contract	ACTIVE	2026 term	ACCT-001
06 LumenWorks Service Agreement	contract	ACTIVE	2026-27 term	ACCT-002
ParcelPilot_Assessment_Data.xlsx	data	DATA	snapshot	row-scoped

Conflict register (what each source says & who wins)

Topic	Default / policy	Contract override	Winner
Cancellation fee	No fee <=30min; else INR 250 (SOP v4)	Northstar: no fee ever before pickup	Northstar→ no fee; others→ SOP
Failed-pickup credit	>2h + carrier fault → min(INR 500, 10% fee)	LumenWorks: >4h → flat INR 300 (replaces both)	Per account (see calculators)
Credit cap	individual > INR 1,000 needs manager approval	Northstar: INR 5,000 monthly aggregate cap	Both apply on top
Enterprise P1 SLA	v3: 30 min 24x7 (v2 was 1h)	Northstar: 15 min 24x7	ACCT-001→15m; ACCT-004→v3 30m
Growth SLA + coverage	v3: P1 2 business hrs...	LumenWorks: same numbers + NO weekend/after-hours	Contract governs (coverage differs)
Bulk upload limit	5,000 rows (Growth/Enterprise); KI-208 >~3,000	—	Product guide; 5,000 not 3,000

KEY INSIGHT LumenWorks' SLA numbers equal the v3 Growth defaults, yet the contract still governs because it adds a coverage restriction (no weekend / after-hours). 'Numbers match' is not 'same source'.

5. Data model & the frozen clock

Workbook → SQLite (typed). 4 accounts, 6 orders, 7 real tickets (+10 synthetic, analytics-only). account_id is the hub; every scoped query filters by the session account.

Table	Key columns	Notes
accounts	account_id PK, plan, contract_file, premium_support	ACCT-001 Northstar/Ent, 002 LumenWorks/Growth, 003 Beacon/Std, 004 Axis/Ent
orders	order_id PK, account_id FK, status, booked_at, pickup_window_end, carrier_fault, shipment_fee_inr	status BOOKED/PICKED_UP/DELIVERED; pickup_actual_at null while unpicked
tickets	ticket_id PK, account_id FK, status, historical_resolution, is_synthetic	historical_resolution may be WRONG; is_synthetic=1 never reaches the agent

- Frozen clock: SNAPSHOT_AT = 2026-08-16 11:00 IST lives in config and is the only 'now'. A test greps src/scripts/evals for datetime.now(/date.today(/time.time(and fails if found.
- Data-quality traps handled: customer_fault has zero variance (still read per-row); pickup delay for unpicked orders is measured to the snapshot; timestamps assumed Asia/Kolkata.

6. Deterministic calculators (src/domain) — Rule 6

Pure functions: primitives in, a typed result (with citations) out. The tool layer fetches the DB row, resolves precedence, then calls these. The model reports the result — it never computes.

CALC-1 Cancellation fee

- DRAFT → free. PICKED_UP → not cancellable (return-to-origin). DELIVERED → not cancellable.
- BOOKED: contract waiver → INR 0 any time; else ≤30min → 0, >30min → INR 250.

CALC-2 Failed-pickup service credit

- delay = (pickup_actual_at or snapshot) - pickup_window_end.
- Default (SOP): delay > 2h AND carrier_fault AND NOT customer_fault → min(INR 500, 10% fee).
- LumenWorks: delay > 4h → flat INR 300 (replaces threshold & amount).
- Northstar: SOP formula + INR 5,000 monthly aggregate cap (clamped).
- Individual credit > INR 1,000 → manager-approval flag. Unknown fault/timing → needs_verification (don't promise).

CALC-3 First-response SLA + breach

- Only clock-time 24x7 targets are exactly computable (e.g. Northstar P1 15 min).
- Business-hours / business-day targets → computable=False (indeterminate): the docs never define a business calendar, so we don't guess — a deliberate trust signal.

CALC-4 Bulk-upload advisory

- Standard → not available. Growth/Enterprise → supported to 5,000; >~3,000 attaches the KI-208 advisory + split workaround.

WHY DETERMINISTIC Money/eligibility in tested Python (not the LLM) is what makes answers auditable and stops 'confidently wrong' arithmetic. Policy thresholds are transcribed into cited dataclasses (policy.py) — not invented, and the raw text stays the authority the agent quotes.

7. Retrieval, LLM client, access control

Retrieval (src/retrieval)

- PDFs → page-marked text → section chunks (~24) tagged with tier/status/bound-account.
- Hybrid score = $0.5 * \text{minmax}(\text{cosine of fastembed}) + 0.5 * \text{minmax}(\text{BM25})$. No embeddings API.
- Gating: deprecated excluded unless the query asks 'what changed / v2'; contract chunks visible only to their account (or internal_ops).

LLM client (src/llm/client.py) — the ONLY SDK caller

- Neutral messages/tools adapted to each provider; returns text and/or tool calls.
- Groq primary (clean tool calling). Gemini fallback — a 3.x 'thinking' model that requires replaying a thought_signature on tool-call history (client captures/replays it).
- Automatic fallback on primary error — a live-demo safety net.

Access control (src/data/repository.py) — Rule 4

- Session(role, account_id) from a bearer token (auth.py). Customer pinned to own account.
- Another account's order/ticket returns None (not an error that leaks existence).
- The model can pass an order_id but never an account_id — it cannot widen scope.

```
# demo tokens
cust-acct-001..004 -> customer, that account
ops -> internal_ops (broad read)
```

8. Agent loop & proposal gating

- Hand-written loop (Rule 8): call LLM with tool schemas; run tool calls; feed results back; repeat until a text answer or the 8-iteration cap.
- iter_agent is a generator → /chat/stream emits SSE (tool_call / tool_result / final) for live traces.
- System prompt states the rules, but enforcement is in code (repository, calculators, proposal store).
- State-changing tools return a pending Proposal; execution only via POST /proposals/{id}/confirm, scoped to the owning account (Rule 5). In-memory store (mock actions, per the brief).

9. Problem 1 — proactive detection (Ops Board)

Internal-only analytics (/ops/signals) over ALL tickets (real + synthetic), measured at the snapshot.

- SLA watch: every open ticket's target + breach (24x7 exact; business-hours indeterminate).
- Clusters: tickets grouped by known-issue signature (KI-208 bulk upload, KI-211 SwiftShip).
- Multi-customer: the same signature across ≥ 2 accounts at once.
- Spikes: surge in the 24h before the snapshot vs baseline.
- Synthetic tickets (is_synthetic=1) add signal but are ISOLATED — the customer agent reads real data only.

10. API, frontend, hosting, tests

Endpoint	Purpose
GET /health	status + active provider + key presence (debug)
GET /me	session role + account_id
POST /chat	run agent (collected result)
POST /chat/stream	SSE: streamed tool_call / tool_result / final

Endpoint	Purpose
GET /ops/signals	Problem-1 dashboard (internal only)
GET /proposals; POST /proposals/{id}/confirm cancel	list / execute / drop a pending action

- Frontend: React/Vite; persona switch (=token), streamed tool-trace chips (expandable JSON+citation), markdown answers, confirmation cards, ops board.
- Hosting: multi-stage Dockerfile builds React, bakes extracted data + SQLite + embedding model (offline runtime), serves everything on one origin; render.yaml blueprint.
- Tests: 46 pytest (calculators, precedence, access-control isolation, retrieval gating, ops signals, API auth/proposals) + the datetime grep test; ruff clean.

DEPLOY GOTCHA If the app answers with a Gemini 503, LLM_PROVIDER is not set to groq on the host (it defaults there) or GROQ_API_KEY is missing. Set LLM_PROVIDER=groq + both keys; /health should show llm_provider=groq, groq_key_set=true.

11. Key decisions & trade-offs (be ready to defend these)

Decision	Why	Trade-off / rejected
Deterministic calculators + precedence resolver	Model must not do math or decide eligibility → auditable, testable, not confidently wrong.	More code vs letting the LLM reason freely (rejected: unverifiable).
Local embeddings (no API)	Corpus ~24 chunks: local is faster, offline, container-safe.	~130MB model in image vs a hosted embedder (no benefit at this size).
Groq primary, Gemini fallback	Groq clean tool calls; Gemini 3.x hit 503/429 + needs signature replay.	Reversible via one env var; fallback is the demo safety net.
No agent framework	Transparent, few deps, full control of the loop; shows fundamentals.	Hand-written vs LangChain conveniences.
Access control in data layer	A customer cannot reach another account even if the model is coaxed.	Slightly more plumbing than prompt-only (which is unsafe).
Frozen clock + grep test	Reproducible, snapshot-consistent time-based answers.	Bans wall clock repo-wide.
Synthetic tickets isolated by flag	Demonstrate Problem 1 without polluting the authority base.	Extra column + dual read path.

12. Known limitations & ‘what I’d build next’

Documented limitations (not hidden)

- Business-hours SLA targets are ‘indeterminate’ (source docs never define the calendar).
- Northstar monthly cap logic is tested but month-to-date is not persisted (no credit ledger).
- Proposals + auth are in-memory / mock (allowed by the brief).

Prioritised roadmap (protect core → close loop → scale)

- Tier 1: answer-level eval harness + CI gate; real action write-back + audit log; business-hours calendar.
- Tier 2: one-click actions from the Ops Board; credit ledger + cap enforcement; answer feedback loop.
- Tier 3: retrieval reranking + doc versioning; provider observability + smart routing; real auth + isolation tests; multi-turn memory.

ONE METRIC Grounded-resolution rate — share of answers that cite a governing source AND match the deterministic resolver, with a hard sub-goal of ~0 incorrect-authority rate (never basing an answer on a deprecated/historical source). Companion: appropriate-escalation rate.

13. Likely interview questions & crisp answers

Q. How do you stop the agent being confidently wrong?

A. Four layers: a deterministic precedence resolver picks the governing source; deterministic calculators (not the LLM) do all money/SLA math; every answer cites its source; and historical/deprecated sources are never used as authority. Unknown inputs → escalate, not guess.

Q. Why no agent framework?

A. A hand-written loop is transparent, has few dependencies, and gives full control over tool schemas, the 8-iteration cap, and streaming. It also demonstrates fundamentals rather than hiding them.

Q. How is access control enforced, and why not in the prompt?

A. In the repository (data layer). The Session's account_id comes from a server-side bearer token; a customer query is physically pinned to that account and another account's row returns None. Prompt-only control is bypassable; data-layer control is not.

Q. Why local embeddings instead of an embeddings API?

A. The corpus is ~24 chunks, so embedding quality differences are negligible; local fastembed adds no latency, no keys, no rate limits, and works offline inside the container. Groq has no embeddings endpoint anyway.

Q. Why Groq primary over Gemini?

A. Empirical: Groq gpt-oss-120b did clean tool round-trips first try; Gemini 3.x flash hit 503/429 under load and, being a thinking model, rejected tool history missing a thought_signature. Gemini is a wired-up fallback.

Q. Explain the frozen clock and why it matters.

A. The dataset is a snapshot; the workbook README timestamp is the only 'now'. Using the wall clock would make time-based answers (SLA breach, cancellation window) non-reproducible, so datetime.now/date.today are banned and a test greps for them.

Q. How do the two contracts override policy without leaking across accounts?

A. The precedence resolver scope-gates a contract to its bound account_id before ranking. Northstar's waiver applies only to ACCT-001; ACCT-003 asking the same question falls through to the SOP.

Q. LumenWorks' SLA numbers equal v3 — why does the contract still win?

A. Because it adds a material coverage restriction (no weekend/after-hours). Matching numbers is not the same source; citing v3 would drop that clause.

Q. How is 'confirmation before actions' actually enforced?

A. State-changing tools return a pending Proposal and never write on the first call. Execution is a separate server endpoint (/proposals/{id}/confirm) scoped to the owner — enforced in code, not by asking the model.

Q. How does proactive detection avoid corrupting the authority data?

A. Synthetic tickets carry is_synthetic=1. The repository (what the agent sees) reads real rows only; the ops signals module reads real+synthetic for analytics. Analytics never becomes authority.

Q. What happens when fault or timing is unknown?

A. The credit calculator returns needs_verification and the agent refuses to promise a credit — it explains what's missing and offers to escalate. Same philosophy as the indeterminate SLA case.

Q. What would break at 100x the data, and what would you change?

A. Naive hybrid search degrades → add section-aware chunking + a reranker + doc versioning. SQLite is fine for moderate scale. The precedence/calculator core is unaffected. Add an eval harness before any model change.

Q. Walk me through a multi-step query.

A. 'Can I cancel ORD-1001 without a fee?' → search_documents (policy) + get_order (scoped) + calc_cancellation_fee (resolves Northstar's waiver) → a cited answer, all in one streamed turn.

Generated for interview preparation. All facts reflect the implemented project (46 tests passing, deployed on Render). See docs/SOURCE_MAP.md, ARCHITECTURE_NOTE.md, PRODUCT_NOTE.md, DECISIONS.md in the repo for the source-of-truth detail.